

Examen Práctico Complementario: Programación Python

Computación Aplicada

Ciclo Escolar: Semestre Agosto-Diciembre 2026

Alumno:  Josué Julián Ramírez Rosales

correo: josuejrr93@gmail.com

matricula:20202162

Catedrático:  Carlos Héctor Castañeda Ramírez

Datos Generales del Examen

- Ponderación: 30% de la calificación final.
- Tiempo Estimado: 60 minutos.
- Modalidad: Aplicación de consola en Python con asistencia gradual de Inteligencia Artificial.
- Archivo Principal: *p085_SimuladorVentaCombustible.py*

Sección 1: Planteamiento del Problema

Se requiere desarrollar un sistema interactivo de consola para la gestión de una **estación de servicio (gasolinera)**. El programa debe permitir a los operadores realizar cálculos de ventas, proyecciones de rendimiento y categorización de clientes mediante un flujo lógico robusto.

Funcionalidades del Sistema:

1. Menú Principal: El programa debe ejecutarse en un ciclo infinito (while True) que presente cuatro opciones:
 - Venta de Combustible.
 - Simulación de Rendimiento.
 - Clasificador de Cliente.
 - Salir.
2. Módulo de Venta: Solicitar el tipo de combustible, precio por litro y cantidad de litros. Debe validar que los valores sean positivos. El cálculo del total debe presentarse con f-strings, alineación a la derecha y dos decimales.
3. Simulación de Rendimiento: Utilizando un ciclo for, el usuario ingresará un kilometraje inicial y se debe mostrar una tabla proyectada (mes a mes o km

a km) del consumo estimado basado en un factor de rendimiento constante proporcionado por el usuario.

4. Clasificador de Cliente: Mediante estructuras if/elif/else y operadores lógicos, determinar si un cliente es "Regular", "Premium" o "Flotilla" basándose en el volumen de compra mensual (ej. < 100L, 100-500L, > 500L).
5. Finalización: La opción de salida debe terminar el programa de forma limpia.

Requisitos Técnicos Obligatorios

- Variables y Tipos: Uso correcto de int, float y str.
- Operadores Aritméticos: Implementación obligatoria de división entera (//), residuo (%) y potencia (**) en cálculos de simulación o validación.
- Formateo: Uso de f-strings para todas las salidas de datos, cuidando la estética de la consola.
- Control de Flujo: Uso de while con sentencias break y continue, y for con la función range().
- Restricción Estricta: Queda terminantemente prohibido el uso de listas, tuplas, diccionarios, funciones personalizadas o cualquier estructura de datos avanzada no cubierta en el Bloque 1.

Sección 2: Guía de Ejecución Gradual con IA

El examen se divide en cuatro fases. Deberás interactuar con un modelo de lenguaje (LLM) siguiendo este orden:

Fase 0: Detección de Fallas críticas o vacíos en los requerimientos

Se conversó con el agente de AI para ver la viabilidad del proyecto y en conjunto señalar los puntos poco claros, forzados o fatales. Estos fueron los resultados obtenidos.

Antes de programar, con apoyo de la IA, revisé el enunciado y encontré cuatro vacíos que impedían implementarlo sin tomar decisiones propias:

1. **Validación contradictoria:** se pide que el programa no truee ante entradas inesperadas, pero sin `try/except`. Lo resolví validando con métodos de `str`.
2. **Simulación sin definir:** no se indicaban la fórmula ni el número de iteraciones. Fijé 12 meses con consumo = km mensuales / rendimiento.
3. **Límites ambiguos en el clasificador:** no quedaba claro a qué categoría pertenecen exactamente 100 L y 500 L. Definí Premium como el rango de 100 a 500 L, incluidos ambos extremos.

4. **Operadores forzados:** el problema no tenía un uso natural para `//`, `%` y `**`. Les asigné cálculos con sentido: tanques completos y litros sobrantes (`//`, `%`) y el aumento compuesto del precio en la simulación (`**`).

Prompts utilizados

1. *"¿Qué dice en el apartado 1?"*
2. *"¿Qué problemas o qué cosas crees que le falten a este ejercicio para mejorarlo?"*
3. *"En base a estas sugerencias y vacíos existentes, ¿cómo podemos diferenciarnos sin desapegarnos de la rúbrica?"*
4. *"Dame un texto muy breve sobre las fallas más fatales que encontramos, para la bitácora."*

Fase 1: Descomposición y Diseño de Algoritmo

Solicita a la IA que te ayude a estructurar la lógica antes de codificar.

Prompt Sugerido

"Actúa como un experto en Python. Ayúdame a diseñar el pseudocódigo para un menú interactivo que use un ciclo while y una estructura de control para una gasolinera, sin usar funciones ni listas."

Prompts utilizados

1. "Antes de partir con el apartado 2, ¿necesitas que establezcamos otros parámetros?"
2. "2: precios yo los pongo, inicialízalos en 0. 3: está bien 40 L. 7: sin aumento de precio, añade complejidad innecesaria. 8: sin marca de servicio."

Generación del Pseudocódigo del proyecto

INICIO

```
// — CONSTANTES —
CAPACIDAD_TANQUE ← 40          (int, litros)
PRECIO_MAGNA      ← 0.0        (float, precio de lista, lo fija el alumno)
PRECIO_PREMIUM    ← 0.0        (float, precio de lista, lo fija el alumno)
PRECIO_DIESEL     ← 0.0        (float, precio de lista, lo fija el alumno)
TASA_AUMENTO      ← 0.01       (float, 1 % mensual)
MESES              ← 12         (int, periodos relativos)
LIMITE_PREMIUM     ← 100        (int)
LIMITE_FLOTILLA    ← 500        (int)
ANCHO              ← 50         (int)
```

```
// — ACUMULADORES DE SESIÓN —
num_ventas ← 0      litros_vendidos ← 0.0      total_recaudado ← 0.0
```

MIENTRAS VERDADERO:

MOSTRAR menú: 1 Venta | 2 Simulación | 3 Clasificador | 4 Salir
LEER opcion (str)

// ===== OPCIÓN 1: VENTA =====

SI opcion = "1":

// 1.1 Tipo de combustible

MIENTRAS VERDADERO:

 MOSTRAR submenú con precio de lista de cada combustible

 LEER op_tipo

 SI op_tipo = "1": tipo ← "Magna"; precio_lista ← PRECIO_MAGNA;

ROMPER

 SI NO SI "2": tipo ← "Premium"; precio_lista ← PRECIO_PREMIUM;

ROMPER

 SI NO SI "3": tipo ← "Diésel"; precio_lista ← PRECIO_DIESEL;

ROMPER

 MOSTRAR error

// 1.2 Precio (híbrido)

MIENTRAS VERDADERO:

 LEER texto (mostrando "Enter = precio_lista")

 SI texto vacío Y precio_lista > 0: precio ← precio_lista; ROMPER

 SI NO SI texto numérico Y > 0: precio ← float(texto); ROMPER

 MOSTRAR error

// 1.3 Litros

MIENTRAS VERDADERO:

 LEER texto

 SI texto numérico Y > 0: litros ← float(texto); ROMPER

 MOSTRAR error

// 1.4 Cálculos

total ← precio * litros

tanques ← int(litros // CAPACIDAD_TANQUE)

sobranter ← litros % CAPACIDAD_TANQUE

// 1.5 Ticket y acumuladores

MOSTRAR ticket alineado

num_ventas += 1; litros_vendidos += litros; total_recaudado += total

// ===== OPCIÓN 2: SIMULACIÓN =====

SI NO SI opcion = "2":

 Seleccionar combustible (mismo submenú del 1.1) → precio ← precio_lista

 LEER y validar km_inicial (≥ 0), km_mensuales (> 0), rendimiento (> 0)

 litros_total ← 0.0 costo_total ← 0.0

 MOSTRAR encabezado: Mes | Odómetro | Litros | Costo

 PARA mes EN range(1, MESES + 1):

 odometro ← km_inicial + km_mensuales * mes

 litros_mes ← km_mensuales / rendimiento

 precio_mes ← precio * (1 + TASA_AUMENTO) ** (mes - 1)

 costo_mes ← litros_mes * precio_mes

 acumular litros_total y costo_total

 MOSTRAR fila

MOSTRAR totales

```

// ===== OPCIÓN 3: CLASIFICADOR =====
SI NO SI opcion = "3":
    LEER y validar litros_mes (≥ 0)
    SI litros_mes < LIMITE_PREMIUM:
"Regular"
        SI NO SI litros_mes >= LIMITE_PREMIUM Y litros_mes <= LIMITE_FLOTILLA:
"Premium"
            SI NO:
"Flotilla"
                MOSTRAR categoria

// ===== OPCIÓN 4: SALIR =====
SI NO SI opcion = "4":
    SI num_ventas = 0: MOSTRAR "Sin ventas en esta sesión"
    SI NO:
        MOSTRAR resumen de la sesión
    MOSTRAR despedida
    ROMPER

// ===== INVÁLIDA =====
SI NO:
    MOSTRAR "Opción inválida"
    CONTINUAR

FIN

```

Fase 2: Implementación Incremental

Desarrolla los módulos de cálculo uno por uno.

Prompt Sugerido

Escribe el código en Python para calcular el total de una venta de combustible validando que los datos sean float positivos. Usa f-strings para mostrar el resultado con 2 decimales y el símbolo de pesos alineado."

Prompts utilizados

1. "Fase 0, partamos desde justo el inicio: la declaración y comentarios."
2. "Espera, vamos por pasos, quiero hacerlo en conjunto primero."
3. "Fase 2: Implementación Incremental. Desarrolla los módulos de cálculo uno por uno."

Modulo 0 Encabezado y Declaración de variables

En primera instancia se declara el encabezado, variables y constantes del proyecto a realizar

```

# =====
# p085_SimuladorVentaCombustible.py

```

```

# -----
# Examen Práctico Complementario: Programación Python
# Materia:      Computación Aplicada
# Ciclo:        Semestre Agosto-Diciembre 2026
# Catedrático:  Carlos Héctor Castañeda Ramírez
# Alumno:       Josué Julián Ramírez Rosales
# Matrícula:    20202162
# Fecha:        [22/09/26]
# -----
# DESCRIPCIÓN
# Sistema interactivo de consola para una estación de servicio.
# Permite registrar ventas de combustible, simular el rendimiento
# de un vehículo a 12 meses y clasificar clientes según su volumen
# de compra mensual.
# -----
# SUPUESTOS DE DISEÑO (vacíos del enunciado resueltos)
# 1. Tipo de combustible: submenú validado (1 Magna, 2 Premium,
#    3 Diésel) que muestra el precio de lista de cada uno.
# 2. Precios: cada combustible tiene un precio de lista (constante).
#    - Venta: el operador lo acepta con Enter o captura otro.
#      Si el precio de lista es 0.0, se obliga a capturarlo.
#    - Simulación: se usa directamente el precio de lista.
# 3. Capacidad de tanque: 40 L, usada para calcular tanques
#    completos (//) y litros sobrantes (%).
# 4. Simulación: 12 periodos mensuales RELATIVOS (Mes 1 = primer
#    mes proyectado), no meses del calendario.
#    Consumo mensual (L) = km mensuales / rendimiento (km/L).
# 5. Aumento de precio: 1 % mensual compuesto, aplicado con (**).
# 6. Clasificador: Regular < 100 L
#                  Premium 100 L a 500 L (ambos incluidos)
#                  Flotilla > 500 L
# 7. Validación: los datos inválidos se vuelven a pedir en un
#    ciclo while. Se valida con métodos de str (sin try/except).
# 8. Km inicial y litros del clasificador aceptan 0 (>= 0);
#    precio, litros de venta, km mensuales y rendimiento, > 0.
# 9. Al salir se muestra un resumen de las VENTAS de la sesión.
# -----
# RESTRICCIONES RESPETADAS (Bloque 1)
# Sin listas, tuplas, diccionarios ni funciones personalizadas.
# Solo tipos int, float y str; control con while/for/if.
# =====

# ----- CONSTANTES -----
CAPACIDAD_TANQUE = 40      # int    - litros por tanque estándar
PRECIO_MAGNA = 0.0        # float - precio de lista ($/L)
PRECIO_PREMIUM = 0.0      # float - precio de lista ($/L)
PRECIO_DIESEL = 0.0       # float - precio de lista ($/L)
TASA_AUMENTO = 0.01      # float - aumento mensual del precio (1 %)
MESES = 12               # int    - periodos de la simulación
LIMITE_PREMIUM = 100     # int    - litros mínimos para Premium
LIMITE_FLOTILLA = 500    # int    - litros máximos para Premium
ANCHO = 50               # int    - ancho de la consola

```

```

LINEA = "=" * ANCHO          # str - separador principal
SUBLINEA = "-" * ANCHO       # str - separador secundario

# ----- ACUMULADORES DE SESIÓN -----
num_ventas = 0               # int - ventas registradas
litros_vendidos = 0.0        # float - litros vendidos en total
total_recaudado = 0.0        # float - dinero recaudado en total

# ----- VARIABLES DE TRABAJO -----
opcion = ""                  # str - opción del menú principal
tipo = ""                    # str - tipo de combustible
precio_lista = 0.0           # float - precio de lista del combustible elegido
precio = 0.0                  # float - precio por litro aplicado a la venta
litros = 0.0                  # float - litros de la venta
total = 0.0                   # float - total de la venta

# ----- PROGRAMA PRINCIPAL -----

```

Menú principal (ciclo `while True`)

Todos los módulos viven dentro de este ciclo. Primero se descarta la opción inválida con `continue` (cláusula de guarda) y después cada opción válida entra a su módulo con `if` / `elif`.

```

while True:
    print()
    print(LINEA)
    print(f"{'ESTACIÓN DE SERVICIO': ^{ANCHO}}")
    print(f"{'Sistema de Venta de Combustible': ^{ANCHO}}")
    print(LINEA)
    print(" 1. Venta de combustible")
    print(" 2. Simulación de rendimiento")
    print(" 3. Clasificador de cliente")
    print(" 4. Salir")
    print(SUBLINEA)
    opcion = input(" Elige una opción (1-4): ").strip()

    # Opción inválida: se avisa y se regresa al menú (continue salta
    # la pausa del final del ciclo)
    if opcion != "1" and opcion != "2" and opcion != "3" and opcion != "4":
        print(" [!] Opción inválida. Elige un número del 1 al 4.")
        continue

```

Ejemplo del menú con una opción inválida:

```

=====
ESTACIÓN DE SERVICIO
Sistema de Venta de Combustible
=====

```

1. Venta de combustible
 2. Simulación de rendimiento
 3. Clasificador de cliente
 4. Salir
-

Elige una opción (1-4): 7

[!] Opción inválida. Elige un número del 1 al 4.

Una vez establecidas las rubricas, constantes y variables del problema se procede a la elaboración del primer modulo del programa

Modulo 1 Selección, Calculo y Venta de Combustible

```
# ===== MÓDULO 1: VENTA DE COMBUSTIBLE =====
if opcion == "1":
    print()
    print(LINEA)
    print(f"'VENTA DE COMBUSTIBLE':^{ANCHO}")
    print(LINEA)

# 1.1 Selección del tipo de combustible (asigna el precio de lista)
while True:
    print(f"  1. Magna      ${PRECIO_MAGNA:>8,.2f} /L")
    print(f"  2. Premium     ${PRECIO_PREMIUM:>8,.2f} /L")
    print(f"  3. Diésel       ${PRECIO_DIESEL:>8,.2f} /L")
    op_tipo = input("  Selecciona el combustible (1-3): ").strip()
    if op_tipo == "1":
        tipo = "Magna"
        precio_lista = PRECIO_MAGNA
        break
    elif op_tipo == "2":
        tipo = "Premium"
        precio_lista = PRECIO_PREMIUM
        break
    elif op_tipo == "3":
        tipo = "Diésel"
        precio_lista = PRECIO_DIESEL
        break
    print("  [!] Opción inválida. Elige 1, 2 o 3.")

# 1.2 Precio por litro (Enter = precio de lista; o se captura otro)
while True:
    texto = input(f"  Precio por litro [Enter = ${precio_lista:,.2f}]: ").strip()
    if texto == "" and precio_lista > 0:
        precio = precio_lista
        break
    elif texto.replace(".", "", 1).isdigit() and float(texto) > 0:
        precio = float(texto)
        break
    print("  [!] Ingresa un número mayor a 0 (ej. 23.50).")
```



```

# 1.3 Captura y validación de los litros a vender
while True:
    texto = input("  Litros a cargar: ").strip()
    if texto.replace(".", "", 1).isdigit() and float(texto) > 0:
        litros = float(texto)
        break
    print("  [!] Ingresa una cantidad mayor a 0 (ej. 35.5).")

# 1.4 Cálculos de la venta
total = precio * litros                                # float - importe total
tanques = int(litros // CAPACIDAD_TANQUE)              # int    - tanques completos
sobrante = litros % CAPACIDAD_TANQUE                  # float - litros que sobran

# 1.5 Ticket de venta
print()
print(LINEA)
print(f"{'TICKET DE VENTA': ^{ANCHO}}")
print(LINEA)
print(f"  {'Combustible:': <20}{tipo:>28}")
print(f"  {'Precio por litro:': <20}${precio:>27,.2f}")
print(f"  {'Litros cargados:': <20}{litros:>26,.2f} L")
print(SUBLINEA)
print(f"  {'Tanques de ' + str(CAPACIDAD_TANQUE) + ' L': <20}{tanques:>28}")
print(f"  {'Litros sobrantes:': <20}{sobrante:>26,.2f} L")
print(SUBLINEA)
print(f"  {'TOTAL A PAGAR:': <20}${total:>27,.2f}")
print(LINEA)

# Actualización de acumuladores de sesión
num_ventas += 1
litros_vendidos += litros
total_recaudado += total

```

Ejemplo del ticket de venta del combustible:

```

=====
                          VENTA DE COMBUSTIBLE
=====
1. Magna      $    24.50 /L
2. Premium   $    26.30 /L
3. Diésel    $    26.80 /L
Selecciona el combustible (1-3): 1
Precio por litro [Enter = $24.50]:
Litros a cargar: 130.5

=====
                          TICKET DE VENTA
=====
Combustible:                               Magna
Precio por litro:  $                       24.50
Litros cargados:                               130.50 L

```

Tanques de 40 L:	3
Litros sobrantes:	10.50 L

TOTAL A PAGAR:	\$ 3,197.25
----------------	-------------

Presiona Enter para volver al menú...

Modulo 2 Simulación de Rendimiento Combustible

```
# ===== MÓDULO 2: SIMULACIÓN DE RENDIMIENTO =====
elif opcion == "2":
    print()
    print(LINEA)
    print(f"'SIMULACIÓN DE RENDIMIENTO':^{ANCHO}")
    print(LINEA)

# 2.1 Selección del combustible (mismo submenú del 1.1)
while True:
    print(f"  1. Magna      ${PRECIO_MAGNA:>8,.2f} /L")
    print(f"  2. Premium    ${PRECIO_PREMIUM:>8,.2f} /L")
    print(f"  3. Diésel      ${PRECIO_DIESEL:>8,.2f} /L")
    op_tipo = input("  Selecciona el combustible (1-3): ").strip()
    if op_tipo == "1":
        tipo = "Magna"
        precio_lista = PRECIO_MAGNA
        break
    elif op_tipo == "2":
        tipo = "Premium"
        precio_lista = PRECIO_PREMIUM
        break
    elif op_tipo == "3":
        tipo = "Diésel"
        precio_lista = PRECIO_DIESEL
        break
    print("  [!] Opción inválida. Elige 1, 2 o 3.")

# Se usa el precio de lista; si no está definido (0.0), se pide
precio = precio_lista
while precio <= 0:
    texto = input("  Sin precio de lista. Precio por litro ($): ").strip()
    if texto.replace(".", "", 1).isdigit() and float(texto) > 0:
        precio = float(texto)
    else:
        print("  [!] Ingresa un número mayor a 0 (ej. 23.50).")

# 2.2 Kilometraje inicial (acepta 0: vehículo nuevo)
while True:
    texto = input("  Kilometraje inicial (km): ").strip()
    if texto.replace(".", "", 1).isdigit():
        km_inicial = float(texto)
```

```

        break
    print("  [!] Ingresa un número igual o mayor a 0.")

# 2.3 Kilómetros recorridos por mes
while True:
    texto = input("  Km recorridos por mes: ").strip()
    if texto.replace(".", "", 1).isdigit() and float(texto) > 0:
        km_mensuales = float(texto)
        break
    print("  [!] Ingresa un número mayor a 0 (ej. 1500).")

# 2.4 Rendimiento del vehículo
while True:
    texto = input("  Rendimiento del vehículo (km/L): ").strip()
    if texto.replace(".", "", 1).isdigit() and float(texto) > 0:
        rendimiento = float(texto)
        break
    print("  [!] Ingresa un número mayor a 0 (ej. 12.5).")

# 2.5 Tabla de proyección a 12 meses
litros_total = 0.0
costo_total = 0.0
print()
print(LINEA)
print(f"{'PROYECCIÓN A ' + str(MESES) + ' MESES - ' +
tipo.upper():^{ANCHO}}")
print(LINEA)
print(f"  {'Mes':>4}{ 'Odómetro':>13}{ 'Litros':>10}{ '$/L':>10}{ 'Costo
$:>11}")
print(SUBLINEA)
for mes in range(1, MESES + 1):
    odometro = km_inicial + km_mensuales * mes
    litros_mes = km_mensuales / rendimiento
    precio_mes = precio * (1 + TASA_AUMENTO) ** (mes - 1)
    costo_mes = litros_mes * precio_mes
    litros_total += litros_mes
    costo_total += costo_mes
    print(f"  {mes:>4}{odometro:>13,.1f}{litros_mes:>10,.2f}"
          f"{precio_mes:>10,.2f}{costo_mes:>11,.2f}")
print(SUBLINEA)
print(f"  {'Kilometraje final:':<20}{odometro:>25,.1f} km")
print(f"  {'Litros totales:':<20}{litros_total:>26,.2f} L")
print(f"  {'COSTO TOTAL:':<20}${costo_total:>27,.2f}")
print(LINEA)

```

Ejemplo de simulación de rendimiento:

1. Magna \$ 24.50 /L
2. Premium \$ 26.30 /L
3. Diésel \$ 26.80 /L
Selecciona el combustible (1-3): 2
Kilometraje inicial (km): 15000
Km recorridos por mes: 1500
Rendimiento del vehículo (km/L): 12.5

PROYECCIÓN A 12 MESES - PREMIUM

Mes	Odómetro	Litros	\$/L	Costo \$
1	16,500.0	120.00	26.30	3,156.00
2	18,000.0	120.00	26.56	3,187.56
3	19,500.0	120.00	26.83	3,219.44
4	21,000.0	120.00	27.10	3,251.63
5	22,500.0	120.00	27.37	3,284.15
6	24,000.0	120.00	27.64	3,316.99
7	25,500.0	120.00	27.92	3,350.16
8	27,000.0	120.00	28.20	3,383.66
9	28,500.0	120.00	28.48	3,417.50
10	30,000.0	120.00	28.76	3,451.67
11	31,500.0	120.00	29.05	3,486.19
12	33,000.0	120.00	29.34	3,521.05
Kilometraje final:				33,000.0 km
Litros totales:				1,440.00 L
COSTO TOTAL:		\$		40,025.98

Modulo 3 Clasificador de Clientes

```
# ===== MÓDULO 3: CLASIFICADOR DE CLIENTE =====
elif opcion == "3":
    print()
    print(LINEA)
    print(f"{'CLASIFICADOR DE CLIENTE':^{ANCHO}}")
    print(LINEA)

# 3.1 Compra mensual del cliente (acepta 0)
while True:
    texto = input(" Litros comprados al mes: ").strip()
    if texto.replace(".", "", 1).isdigit():
        litros_cliente = float(texto)
        break
    print(" [!] Ingresar un número igual o mayor a 0.")

# 3.2 Clasificación con if/elif/else y operador lógico and
if litros_cliente < LIMITE_PREMIUM:
    categoria = "Regular"
elif litros_cliente >= LIMITE_PREMIUM and litros_cliente <=
```

```

LIMITE_FLOTILLA:
    categoria = "Premium"
else:
    categoria = "Flotilla"

# 3.3 Resultado
print(SUBLINEA)
print(f"  {'Compra mensual:':<20}{litros_cliente:>26,.2f} L")
print(f"  {'CATEGORÍA:':<20}{categoria:>28}")
print(SUBLINEA)
print(f"  Regular <{LIMITE_PREMIUM} | Premium {LIMITE_PREMIUM}-
{LIMITE_FLOTILLA}")
      f" | Flotilla >{LIMITE_FLOTILLA} L")
print(LINEA)

```

Ejemplo clasificación de cliente:

CLASIFICADOR DE CLIENTE	
Litros comprados al mes: 500	
Compra mensual:	500.00 L
CATEGORÍA:	Premium
Regular <100 Premium 100-500 Flotilla >500 L	

Modulo 4 Salida y Finalizacion deCodigo

```

# ===== MÓDULO 4: SALIR =====
elif opcion == "4":
    print()
    print(LINEA)
    print(f"{'RESUMEN DE LA SESIÓN':^{{ANCHO}}}")
    print(LINEA)
    if num_ventas == 0:
        print(f"{'No se registraron ventas en esta sesión.':^{{ANCHO}}}")
    else:
        print(f"  {'Ventas realizadas:':<20}{num_ventas:>28}")
        print(f"  {'Litros vendidos:':<20}{litros_vendidos:>26,.2f} L")
        print(f"  {'Promedio por venta:':<20}${total_recaudado /
num_ventas:>27,.2f}")
        print(SUBLINEA)
        print(f"  {'TOTAL RECAUDADO:':<20}${total_recaudado:>27,.2f}")
    print(LINEA)
    print(f"{'Gracias por usar el sistema. ¡Hasta pronto!':^{{ANCHO}}}")
    print()
    break

```

```
# Pausa antes de volver al menú
input("\n Presiona Enter para volver al menú...")
```

Ejemplo de finalización de sesión:

RESUMEN DE LA SESIÓN		
Ventas realizadas:		1
Litros vendidos:	130.50	L
Promedio por venta: \$	3,197.25	
TOTAL RECAUDADO: \$ 3,197.25		
Gracias por usar el sistema. ¡Hasta pronto!		

Fase 3: Integración y Flujo de Control

Une las partes en el ciclo principal y asegura el uso de break y continue.

🔗 Prompt Sugerido

"Integra este código en un ciclo while True. Si el usuario elige la opción 4, usa break. Si el usuario ingresa una opción inválida, muestra un error y usa continue para reiniciar el menú."

🔗 Prompt Utilizado

"Revisa la integración de los módulos en el ciclo while True: verifica que la opción 4 use break, que una opción inválida muestre un error y use continue, y explícame cómo interactúan los break de los submenús con el ciclo principal."

3.1 Validación de la opción del menú (cláusula de guarda)

```
if opcion != "1" and opcion != "2" and opcion != "3" and opcion != "4":
    print(" [!] Opción inválida. Elige un número del 1 al 4.")
    continue
```

Primero se descarta lo inválido y después se procesa lo válido. - `continue` regresa al inicio del `while` y **salta la pausa** del final del ciclo. Si el usuario se equivoca, ve el menú de inmediato. - **Se usa** `and` **y no** `or`: basta con que la opción sea distinta de las cuatro válidas. Con `or` la condición siempre sería verdadera y ninguna opción pasaría.

3.2 Los dos tipos de `break`

```
while True:
    ...
    # ← ciclo principal
```

```

if opcion == "1":
    while True:
        ...
        if op_tipo == "1":
            break
        ...
elif opcion == "4":
    ...
    break

```

`break` solo sale del ciclo que lo contiene directamente. El `break` del submenú regresa al módulo de venta. Solo el `break` de la opción 4, en el nivel del ciclo principal, termina el programa.

3.3 Cambios al integrar los módulos

- **Indentación:** cada módulo se movió un nivel adentro (4 espacios), dentro de su `if` / `elif`.
- **Acumuladores fuera del ciclo:** `num_ventas`, `litros_vendidos` y `total_recaudado` se declaran antes del `while`. Si estuvieran adentro, se reiniciarían en cada vuelta y el resumen siempre mostraría 0.
- **Pausa al final del ciclo:** está fuera de la cadena `if/elif`, así que se aplica a los tres módulos sin repetirla.

```

# Pausa antes de volver al menú
input("\n Presiona Enter para volver al menú...")

```

3.4 Prueba de flujo completa

Entrada	Resultado esperado	Obtenido
(vacío), <code>abc</code> , <code>0</code> , <code>5</code>	Error y el menú otra vez, sin pausa	✓
<code>" 1 "</code> (con espacios)	Entra a Venta gracias a <code>.strip()</code>	✓
<code>9</code> en el submenú	Error y el submenú otra vez (no el menú principal)	✓
Venta 1: Magna, Enter, 40 L	\$980.00	✓
Venta 2: Diésel, \$27, 200 L	\$5,400.00	✓
<code>4</code>	Resumen: 2 ventas, 240 L, \$6,380.00, promedio \$3,190.00	✓

Fase 4: Pruebas de Límite y Formato

Refina la salida visual.

🔗 Prompt Sugerido

"Revisa mi código actual. Asegúrate de que la simulación de rendimiento use un ciclo for con range() y que los encabezados de las tablas en consola se vean profesionales y alineados."

Prompt utilizado

"Revisa mi código actual. Asegúrate de que la simulación de rendimiento use un ciclo for con range() y que los encabezados de las tablas se vean profesionales y alineados. Luego pruébalo con valores límite: números enormes, decimales mínimos, los bordes del clasificador y caracteres extraños."

Verificación de lo solicitado

- La simulación usa `for mes in range(1, MESES + 1)`. ✓
- Los encabezados de las tablas están alineados a 50 caracteres. ✓
- Las pruebas de límite encontraron **dos fallas graves** (el programa se detenía con error) y **un problema de formato**.

Falla 1: el programa truena con ²

Detectado: `"^".isdigit()` devuelve `True`, porque Python considera el superíndice como dígito, pero `float("^")` lanza `ValueError` y el programa se detiene.

Corrección: usar `.isdecimal()` en lugar de `.isdigit()`. Solo acepta dígitos que `float()` sí puede convertir.

```
# Antes
if texto.replace(".", "", 1).isdigit() and float(texto) > 0:

# Después
if texto.replace(".", "", 1).isdecimal() and float(texto) > 0 and float(texto) <= VALOR_MAXIMO:
```

Falla 2: el programa truena con un número gigante

Detectado: un número de 400 dígitos pasa la validación, pero `float()` lo convierte en `inf` (infinito). Después, `int(inf // 40)` lanza `ValueError` en el ticket. **Corrección:** una constante de tope, aplicada en las 7 validaciones.

```
VALOR_MAXIMO = 1000000 # int - tope de cualquier captura numérica
```

Se agregó como supuesto de diseño en la cabecera:

```
# 10. Tope de captura: ningún dato numérico puede superar 1,000,000.
# Evita que un número gigante se vuelva "inf" y rompa los cálculos.
```


Los mensajes de error ahora indican el tope:

```
print(f" [!] Ingresa un precio mayor a 0 y hasta {VALOR_MAXIMO:,}.")
```

Problema de formato: columnas pegadas en la simulación

Detectado: con valores extremos, los números de la tabla se juntaban y se volvían ilegibles:

```
1  1,099,999.0200,000.00      24.504,900,000.00
```

Corrección: un espacio fijo entre columnas. Con valores normales la tabla sigue midiendo 50 caracteres, y con valores extremos se alarga pero sigue siendo legible.

```
# Antes
print(f" {'Mes':>4}{'Odómetro':>13}{'Litros':>10}{'$/L':>10}{'Costo $':>11}")
print(f" {mes:>4}{odometro:>13,.1f}{litros_mes:>10,.2f}" f"{precio_mes:>10,.2f}
{costo_mes:>11,.2f}")

# Despues
print(f" {'Mes':>3} {'Odómetro':>12} {'Litros':>9} {'$/L':>9} {'Costo $':>11}")
print(f" {mes:>3} {odometro:>12,.1f} {litros_mes:>9,.2f}" f" {precio_mes:>9,.2f}
{costo_mes:>11,.2f}")
```

Resultado con valores extremos:

```
1  1,099,999.0 200,000.00      24.50 4,900,000.00
```

Batería de pruebas (con las correcciones)

Prueba	Entrada	Resultado
Superíndice	²	Rechazado ✓ (antes se detenía con error)
Número gigante	400 dígitos	Rechazado ✓ (antes se detenía con error)
Justo arriba del tope	1000001	Rechazado ✓
Tope exacto	1000000 L × \$1,000,000	Ticket correcto: \$1,000,000,000,000.00 ✓
Decimal mínimo	0.001 L	Aceptado, total \$0.02 ✓
Clasificador	99.99 / 100 / 500 / 500.01 / 0	Regular / Premium / Premium / Flotilla / Regular ✓
Simulación extrema	999,999 km, 100,000 km/mes, 0.5 km/L	Tabla legible, sin errores ✓
Simulación normal	15,000 km, 1,500 km/mes, 12.5 km/L	Tabla de 50 caracteres exactos ✓

Límite conocido: con 0.001 L el ticket muestra "0.00 L", porque el formato redondea a 2 decimales.

El cálculo interno es correcto; es un límite de la presentación, no un error. *Precios de lista usados en la prueba: Magna \$24.50, Premium \$26.30, Diésel \$26.80.*

Sección 3: Entregables

El alumno deberá cargar en la plataforma institucional los siguientes archivos antes del cierre del tiempo establecido:

1. Código Fuente: Un archivo con extensión .py nombrado exactamente *p085_SimuladorVentaCombustible.py* ✓.
2. Bitácora de IA: Un documento PDF nombrado *p085-bitacora-ia.pdf* ✓ que contenga las capturas de pantalla o el historial de los prompts utilizados y una breve reflexión (máximo 5 renglones) sobre cómo la IA ayudó a resolver los errores de lógica.

Reflexión

La elaboración de este proyecto llevo a un tipo de ejecución complementaria con la IA. A la hora de empezar a leer los requerimientos y restricciones técnicas del proyecto, *detecté algunos vacíos e incongruencias* y ahí es donde **la inteligencia artificial** ayudó a detectar esos vacíos del enunciado (límites del clasificador, fórmula de la simulación) y poder encontrar áreas de mejora y oportunidad antes de pasar a la programación. Además establecimos un *canal de comunicación bastante crítico* con las ideas y planteamientos que proponía y las propuestas generaba el agente. Finalmente la elaboración de este proyecto complementa y interioriza los conceptos ya mostrados en clase y reafirma la actual forma colaborativa con la que se trabaja a día de hoy con la AI.